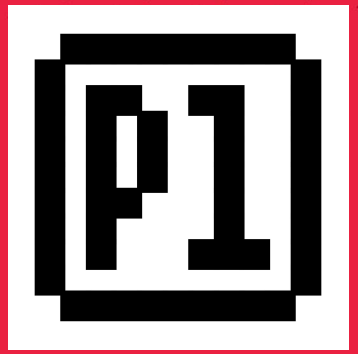
Recursividad



UNRN

Universidad Nacional
de **Río Negro**

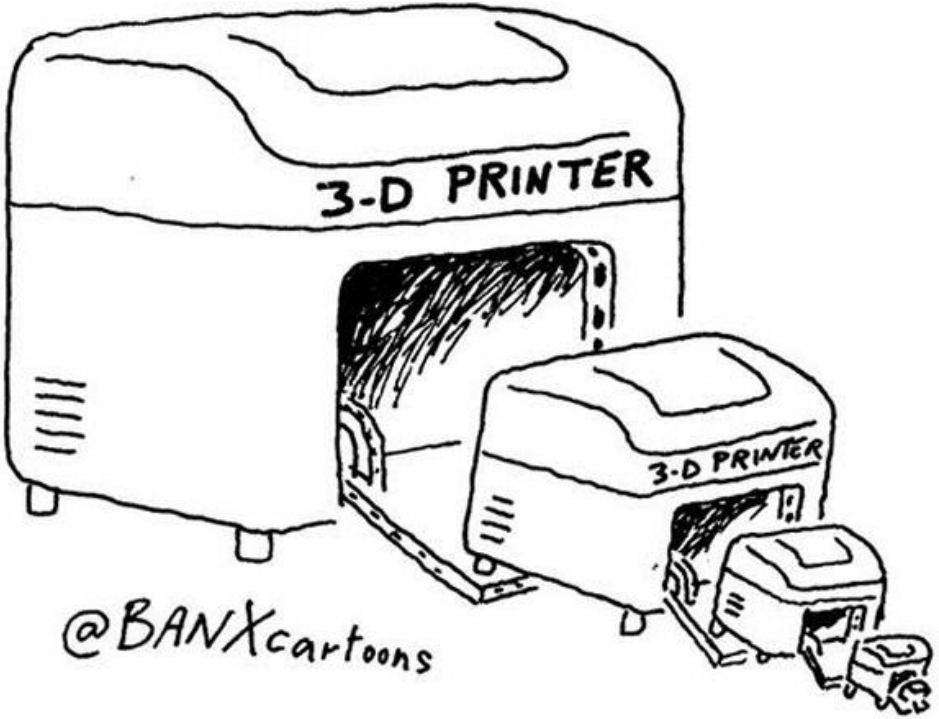
r33

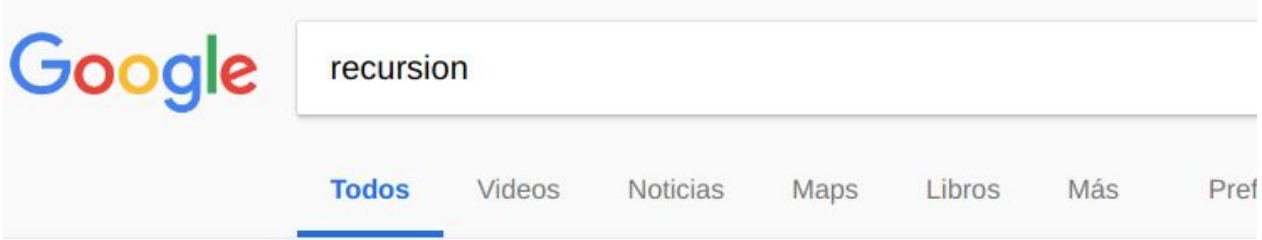
20 / 2026

Recursividad

UNRN Universidad Nacional de Río Negro


sividad


3-D PRINTER
3-D PRINTER
@BANXcartoons


Google
recursion
Todos
Videos
Noticias
Maps
Libros
Más
Pref
Cerca de 6,680,000 resultados (0.28 segundos)
Quizás quisiste decir: recursion
Recurrencia, recursión o recursividad es la forma en la cual se especifica un proceso basado en su propia definición, siendo ésta característica discernible en términos de autorreferencialidad, autopoiesis, fractalidad, o, en otras palabras, construcción a partir de un mismo tipo.
Recursión - Wikipedia, la enciclopedia libre
https://es.wikipedia.org/wiki/Recursión


PAUL
NOTH


SAFELY ENDANGERED
SWEET JESUS, POOH! THAT'S NOT HONEY
SAFELY ENDANGERED
YOU'RE EATING RECURSION

Recursión

La recursión ocurre cuando algo está definido en términos de sí mismo o su tipo.

Estructura de una función recursiva

UNRN Universidad Nacional de Río Negro

Estructura base

```
int funcion(int valor ){
    if ( valor == 0 )
    {
        return valor;
    }
    else
    {
        return funcion( valor - 1 );
    }
}
```

Estructura base

```
int funcion(int valor ){
    if ( valor == 0 )
    {
        return valor;
    }
    else
    {
        return funcion( valor - 1 );
    }
}
```

Caso base

Llamada recursiva

sobre el caso base...

UNRN Universidad Nacional de Río Negro

```
int factorial(int n) {
    // ups, me olvidé el caso base
    return n * factorial(n - 1);
}
```



El caso base es *importante*




¿Preguntas?

Ventajas

El código es intuitivo en su proposito.
La división de un problema en subproblemas más simples es directa.

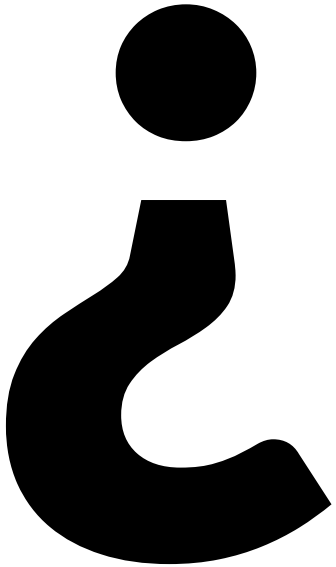
Desventajas

Consumo de memoria elevado[1]

¡Es más fácil caer en un lazo infinito!

Son *generalmente* más lentas[2]

No hay forma de manipular la repetición



Cuando se usa



Ejemplos

Suma

UNRN Universidad Nacional de Río Negro

UNRN Universidad Nacional de Río Negro

La suma de los números entre n y 1

$$\text{suma}(n) = n + n - 1 + \ldots + 1$$

$$\text{suma}(n) = \begin{cases} \text{suma}(1) = 1 & \text{si } n = 0 \\ \text{suma}(n) = n + \text{suma}(n - 1) & \text{si } n > 0 \end{cases}$$

Factorial

UNRN Universidad Nacional de Río Negro

UNRN Universidad Nacional de Río Negro

Algunos ejemplos

$1! = 1 \Rightarrow 1$

$2! = 2 \Rightarrow 2 * 1$

$3! = 6 \Rightarrow 3 * 2 * 1$

$4! = 24 \Rightarrow 4 * 3 * 2 * 1$

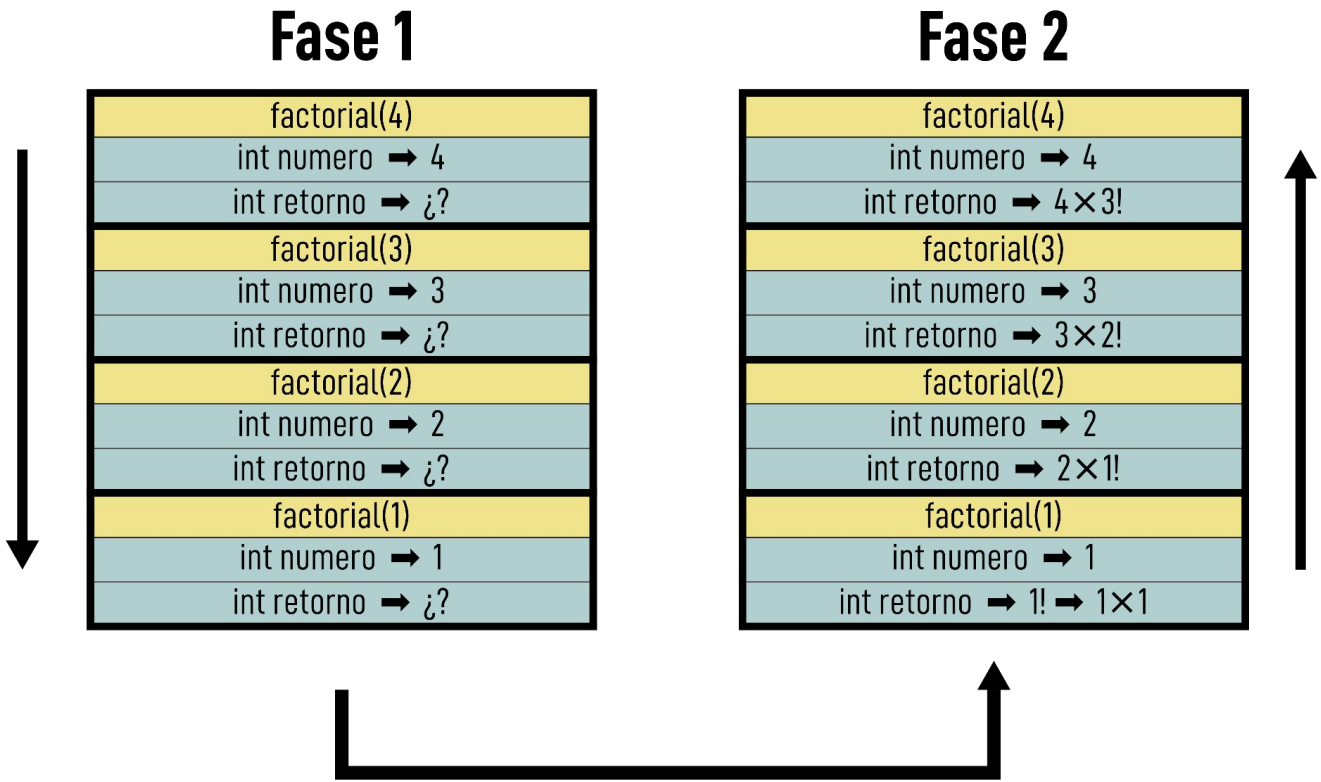
$5! = 120 \Rightarrow 5 * 4 * 3 * 2 * 1$

Definición de Factorial

$$n! = n \times (n - 1) \times (n - 2) \times (n - 3) \times \ldots \times 1$$

$$n! = \prod_{i=1}^{n} i$$

$$n! = \begin{cases} n! = 1 & \text{si } n = 0 \\ n! = n \times (n - 1)! & \text{si } n > 0 \end{cases}$$


Fase 1
factorial(4)
int numero ➡ 4
int retorno ➡ ¿?
factorial(3)
int numero ➡ 3
int retorno ➡ ¿?
factorial(2)
int numero ➡ 2
int retorno ➡ ¿?
factorial(1)
int numero ➡ 1
int retorno ➡ ¿?
Fase 2
factorial(4)
int numero ➡ 4
int retorno ➡ 4×3!
factorial(3)
int numero ➡ 3
int retorno ➡ 3×2!
factorial(2)
int numero ➡ 2
int retorno ➡ 2×1!
factorial(1)
int numero ➡ 1
int retorno ➡ 1! ➡ 1×1

Factorial

```
long factorial(long valor)
{
    if (valor < 1L){
        return 1L;
    }
    else
    {
        return valor * factorial(valor - 1L);
    }
}
```

Implementación iterativa

```
unsigned long factorial(unsigned short valor)
{
    unsigned long i = 1UL;
    unsigned long resultado = 1UL;
    while (i <= valor)
    {
        resultado = resultado * i;
        i = i + 1UL;
    }
    return resultado;
}
```

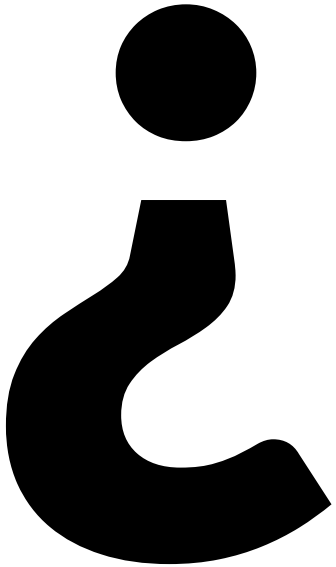

¿Preguntas?

Fibonacci

$$\text{fib(n)} = \begin{cases} 0 & \text{si n} = 0 \\ 1 & \text{si n} = 1 \\ \text{fib(n-1)} + \text{fib(n-2)} & \text{si} > 1 \end{cases}$$

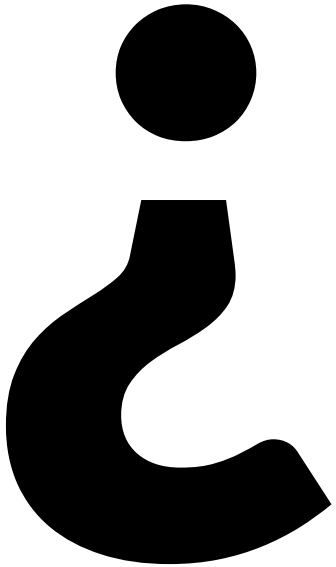
¡Fibonacci recursivo!

```
long fibonacci(int termino)
{
    if (termino == 0) {
        return 0L;
    } else if (termino == 1) {
        return 1L;
    } else {
        return fibonacci(termino - 1) + fibonacci(termino - 2);
    }
}
```



Cuantas llamadas a la función son necesarias





Cuanta memoria consume el algoritmo

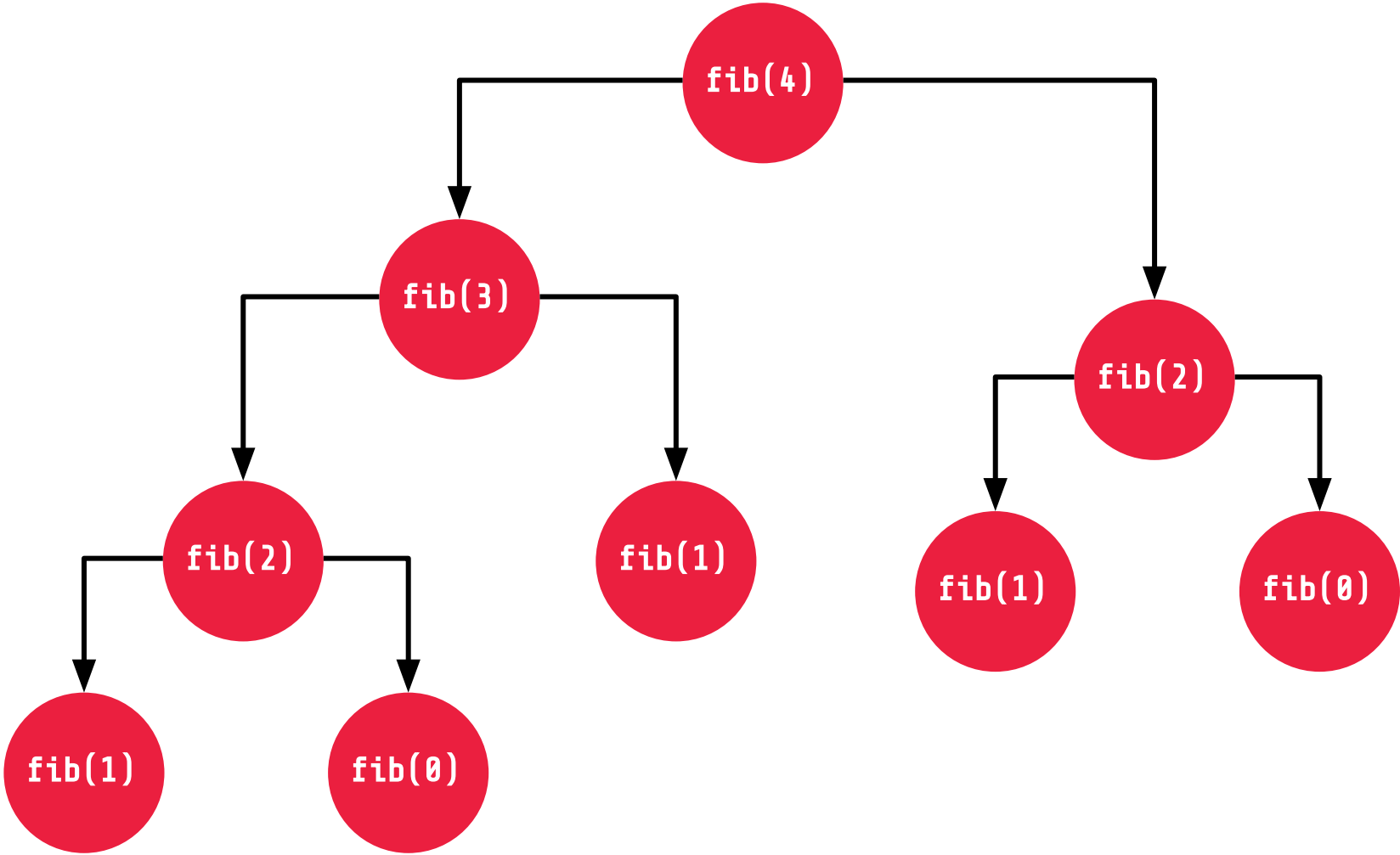


$O(2^n)$



El nudo

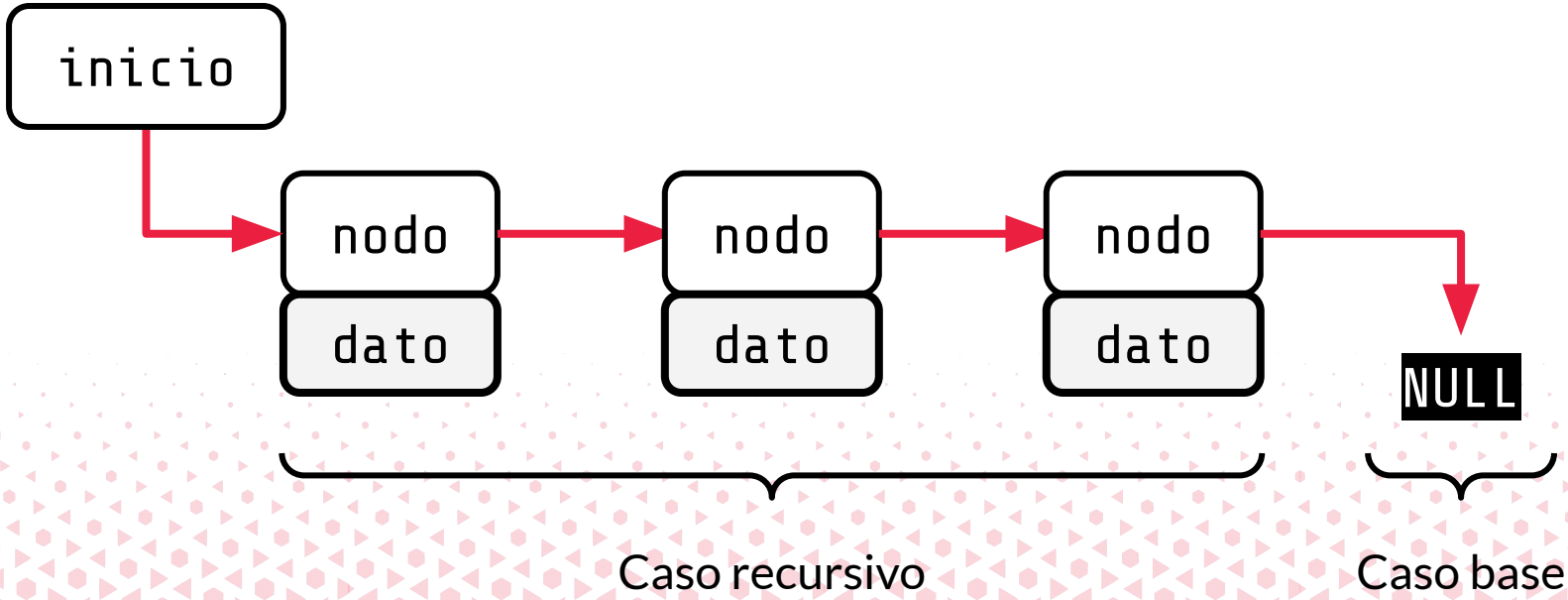
```
long fibonacci(int termino)
{
    if (termino == 0) {
        return 0L;
    } else if (termino == 1) {
        return 1L;
    } else {
        return fibonacci(termino - 1) + fibonacci(termino - 2);
    }
}
```


fib(4)
fib(3)
fib(2)
fib(1)
fib(2)
fib(1)
fib(0)
fib(1)
fib(0)

Listas enlazadas

UNRN Universidad Nacional de Río Negro

Lista enlazada



Recorriendo la lista

```
void imprime_lista(nodo_t *nodo)
{
    if(nodo != NULL)
    {
       printf("%d, ", nodo->valor);
       imprime_lista(nodo->siguiente);
    }
}
```

Una lista se define en términos de sí misma :-D

```
typedef struct nodo
{
    struct nodo *siguiente;
    int valor;
}nodo_t;
```


¿Preguntas?

Divide y conquista

UNRN Universidad Nacional de Río Negro

¿Qué es?

UNRN Universidad Nacional de Río Negro

Un poco de historia

UNRN Universidad Nacional de Río Negro

Pasos

UNRN Universidad Nacional de Río Negro

Dividir

Para aplicarlo, un problema dividido debe ser igual que el problema sin dividir.

Conquistar

- Si el problema es complejo[1], dividirlo
- Si el problema es simple[2], resolverlo

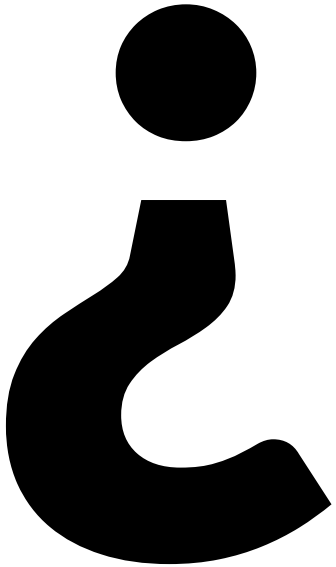
Combinar

los subproblemas simples


¿Preguntas?

Ventajas

Desventajas



Cuando aplicarlo

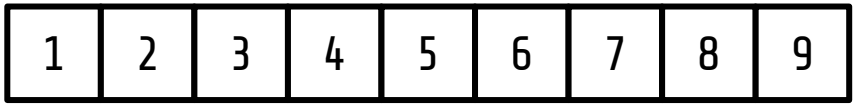



¿Preguntas?

Ejemplos Recursividad + DyC

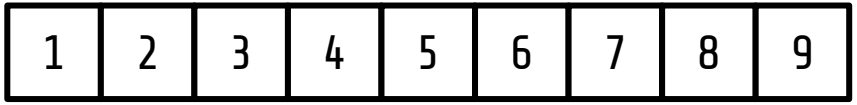
UNRN Universidad Nacional de Río Negro

Suma de elementos recursiva



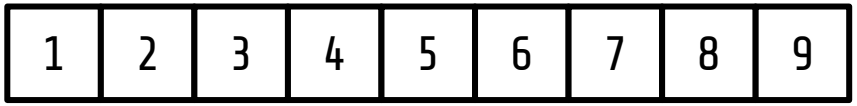

```
int suma(int arreglo[], size_t largo);
```

¡Aprovechando aritmética de punteros!




```
int suma(int arreglo[], size_t largo){
    if (largo <= 0) {
        return 0;
    } else if (largo == 1){
        return arreglo[0];
    } else {
        return arreglo[0] + suma(arreglo + 1, largo - 1);
    }
}
```

Suma de elementos por división y conquista




```
int suma_DyC(int arr[], int inicio, int fin);
```

Implementación de suma por DyC

```
int suma_DyC(int arr[], int inicio, int fin) {
    if (inicio == fin) {
        return arr[inicio];
    }
    int medio = inicio + (fin - inicio) / 2;

    int izquierda = suma_DyC(arr, inicio, medio);
    int derecha = suma_DyC(arr, medio + 1, fin);

    return izquierda + derecha;
}
```

Base

División

Conquista


¿Preguntas?

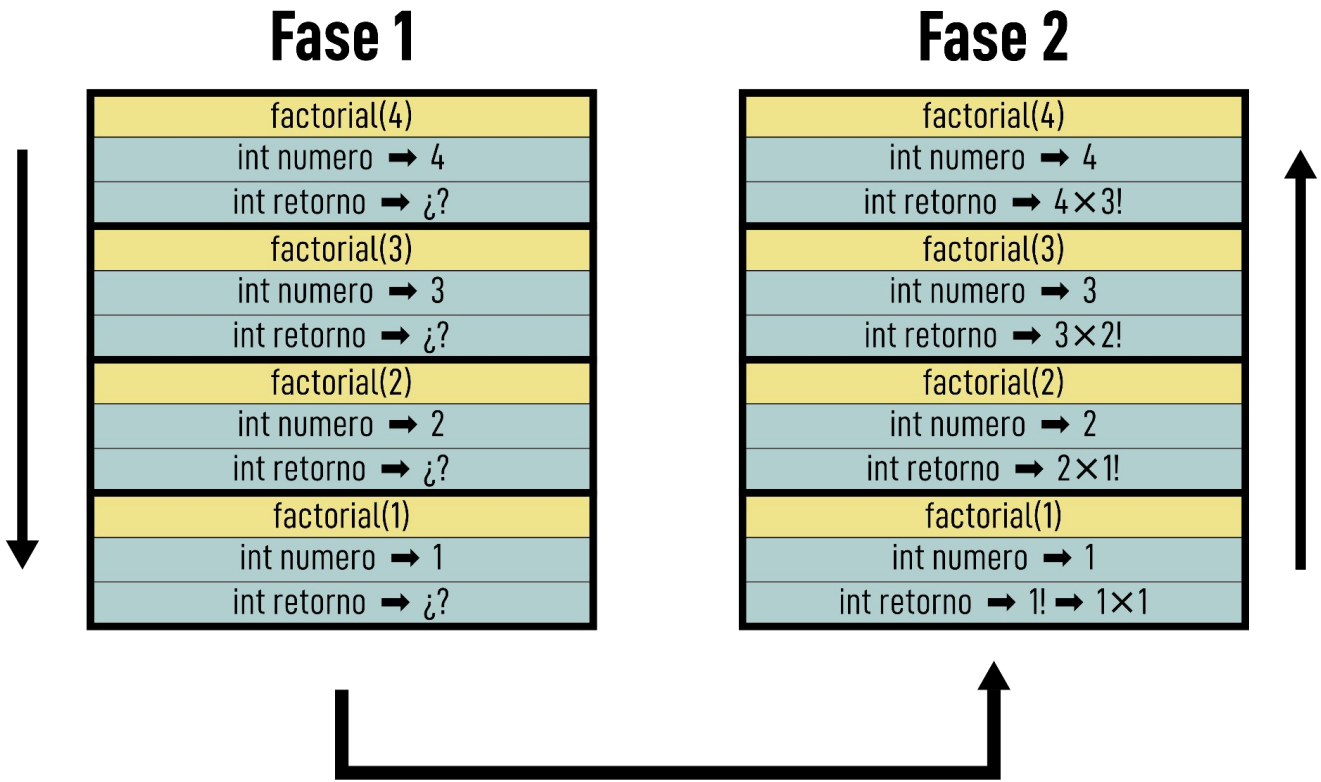
Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

Funciones recursivas

UNRN Universidad Nacional de Río Negro

```
int factorial(int numero)
{
    int retorno;
    if (numero > 1)
    {
        retorno = numero * factorial(numero - 1);
    }
    else
    {
        retorno = 1 * 1;
    }
    return retorno;
}
```


Fase 1
factorial(4)
int numero ➡ 4
int retorno ➡ ¿?
factorial(3)
int numero ➡ 3
int retorno ➡ ¿?
factorial(2)
int numero ➡ 2
int retorno ➡ ¿?
factorial(1)
int numero ➡ 1
int retorno ➡ ¿?
Fase 2
factorial(4)
int numero ➡ 4
int retorno ➡ 4×3!
factorial(3)
int numero ➡ 3
int retorno ➡ 3×2!
factorial(2)
int numero ➡ 2
int retorno ➡ 2×1!
factorial(1)
int numero ➡ 1
int retorno ➡ 1! ➡ 1×1

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar